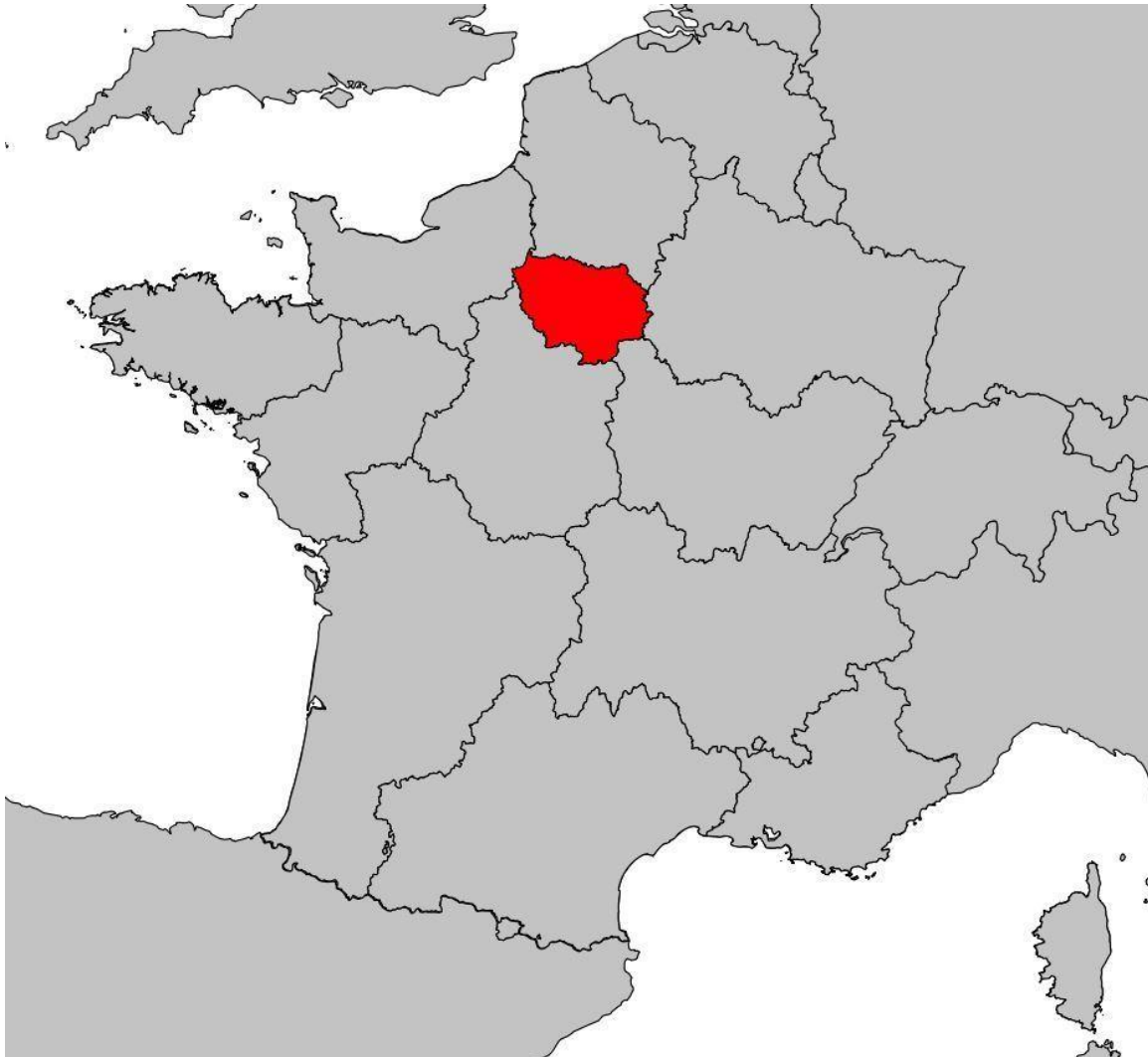


Rapport SAÉ — Échantillonnage et Estimation

Île-de-France



Evan Caillard - Karim Draoui

BUT Science des Données 1
2025 - 2026

Table des matières

Introduction3

Partie 1 — Estimation de la population d’Île-de-France4

 1.1 Sondage Aléatoire Simple (SAS)4

 1.2 Sondage Stratifié7

 Conclusion de la Partie 1..... 11

Partie 2 — Enquête Sport & Étudiants..... 12

 2.1 Présentation des données 12

 2.2 Tableaux croisés et tests du Khi-deux 12

 2.3 V de Cramer et tableau récapitulatif 13

 2.4 Commentaires et conclusion..... 15

Conclusion générale 15

CODE R : 16

Introduction

Dans ce travail, on va chercher à estimer le nombre total d'habitants d'une région de France à partir d'un échantillon de communes. La région que nous avons choisie est l'**Île-de-France**. L'idée est simple : plutôt que de compter tous les habitants de toutes les communes, on va tirer un échantillon de communes d'Ile-de-France, calculer une estimation, et voir à quel point on est proche de la vraie valeur.

Pour mesurer la précision de cette estimation, on va utiliser des **intervalles de confiance**. On va également comparer deux méthodes d'échantillonnage : le sondage aléatoire simple et le sondage stratifié, pour voir laquelle donne de meilleurs résultats.

La deuxième partie du rapport porte sur une enquête sur la pratique sportive des étudiants, où on cherchera à voir si certaines variables comme le sexe ou le niveau d'études sont liées à la pratique du sport.

Partie 1 — Estimation de la population d'Île-de-France

1.1 Sondage Aléatoire Simple (SAS)

Présentation des données

La première étape était de charger les données et de filtrer uniquement les communes d'Île-de-France.

```
# On importe le fichier CSV (après suppression des 6 premières lignes)
# dans Excel et sauvegarde au format .csv)
population <- read.csv2("population_francaise_communes.csv", encoding = "latin1")

# On filtre uniquement les communes de la région Île-de-France
# et on sélectionne les 3 colonnes utiles : Code.département, Commune, Population.totale

donnees <- population[population$Nom.de.la.région == "Île-de-France",
                      c("Code.département", "Commune", "Population.totale")]
```

On charge le fichier CSV et on garde uniquement les lignes correspondant à l'Île-de-France.

```
# Affichage des 6 premières lignes
head(donnees)
```

Code.département	Commune	Population.totale
75	Paris 1er Arrondissement	16149
75	Paris 2e Arrondissement	21277
75	Paris 3e Arrondissement	33651
75	Paris 4e Arrondissement	29326
75	Paris 5e Arrondissement	58050
75	Paris 6e Arrondissement	40452

Cette capture montre les premières lignes de notre jeu de données. On voit bien le nom des communes, leur département et leur population.

Une fois les données prêtes, on a calculé deux valeurs importantes :

- **N** : le nombre total de communes en Île-de-France
 - **T_vrai** : la population totale réelle, qu'on calculera en faisant la somme de toutes les populations
-

```
# On crée le vecteur U qui stockera toutes les communes
U <- donnees$Commune

# Nombre total N de communes en île-de-France
N <- length(U)
cat("Nombre total de communes en île-de-France : N =", N, "\n")

# T est la somme de la population totale de toutes les communes
# d'île-de-France : c'est la vraie valeur que l'on cherchera à estimer

donnees$Population.totale <- as.numeric(gsub(" ", "", donnees$Population.totale))
T_vrai <- sum(donnees$Population.totale)
cat("Nombre exact d'habitants en île-de-France : T =", T_vrai, "\n")
```

Principe et résultats sur 10 tirages

Le principe du SAS est simple : on tire aléatoirement **n communes** (en l'occurrence $n = 100$) parmi les N , et on estime la population totale à partir de cet échantillon. La formule utilisée est :

$$T_{\text{estime}} = N \times \text{moyenne}(\text{échantillon})$$

On a ensuite répété ce tirage **10 fois** pour voir comment variait l'estimation d'un tirage à l'autre, et on a calculé à chaque fois un **intervalle de confiance à 95%**.

La fonction de tirage SAS et le calcul de T_{estime} + IDC

```
donnees$Population.totale <- as.numeric(gsub(" ", "", donnees$Population.totale))
T_vrai <- sum(donnees$Population.totale)
cat("Nombre exact d'habitants en île-de-France : T =", T_vrai, "\n")

resultats <- data.frame(
  Population_reelle = numeric(),
  Population_estimee = numeric(),
  Marge_erreur = numeric(),
  IDC_bas = numeric(),
  IDC_haut = numeric()
)
```

La boucle des 10 tirages

```

for (i in 1:10) {

# Taille de l'échantillon
n <- 100

# Tirage aléatoire simple (SAS) sans remise de n=100 communes
# parmi les N communes de la population U
E <- sample(1:nrow(donnees), n)

# Création de la table "donnees1" :
# On récupère les données des communes sélectionnées
# avec leurs 3 colonnes : Commune, Code.département, Population.totale

donnees1 <- donnees[E,
                    c("Code.département", "Commune", "Population.totale")]

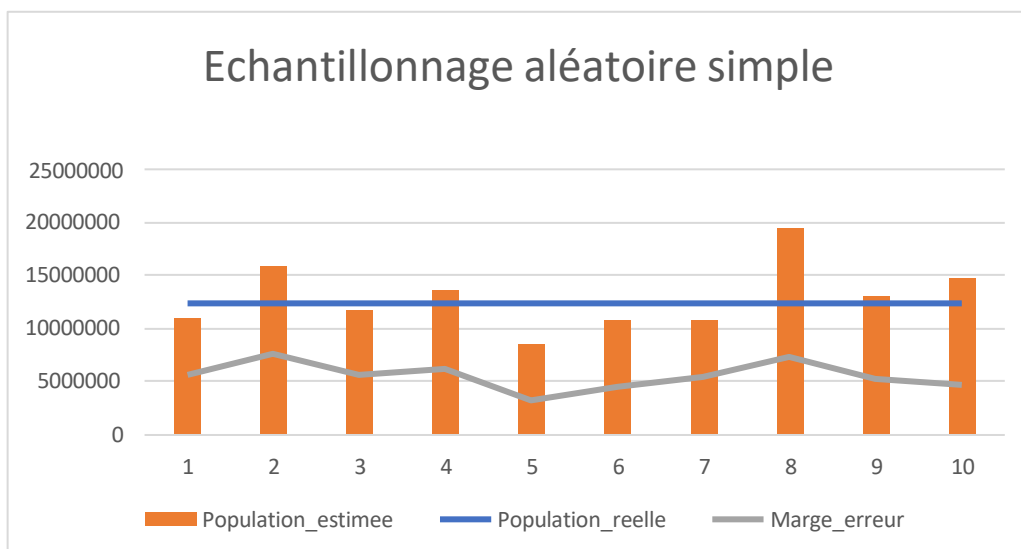
...

new.row <- data.frame(
  Population_reelle = T_vrai,
  Population_estimee = round(T_est),
  Marge_erreur = round(marge),
  IDC_bas = idc_T[1],
  IDC_haut = idc_T[2]
)

resultats <- rbind(resultats, new.row)
}

```

Le graphique des 10 résultats du Sondage Aléatoire Simple



Ce graphique permet de visualiser les 10 estimations. On voit bien que les intervalles sont assez larges, ce qui traduit une certaine imprécision du SAS.

Analyse

On peut remarquer que les estimations varient assez fortement d'un tirage à l'autre. Cela s'explique par le fait que la population des communes en Île-de-France est très **hétérogène** : certaines communes comme Paris ont plusieurs millions d'habitants, alors que d'autres n'en ont que quelques centaines. Le SAS ne tient pas compte de cette hétérogénéité, ce qui explique les grandes variations qu'on observe.

1.2 Sondage Stratifié

Construction des strates

Pour améliorer la précision, on va utiliser un **sondage stratifié**. L'idée est de découper les communes en groupes (appelés strates) selon leur taille, et de tirer un échantillon dans chaque groupe séparément. Ainsi, on s'assure que toutes les catégories de communes sont bien représentées.

On commence avec **4 strates** basées sur les quartiles de population.

Création des 4 strates

```
# Création de 4 strates à partir des quantiles
donnees$strate <- cut(
  donnees$Population.totale,
  breaks = quantile(donnees$Population.totale, probs = seq(0, 1, 0.25)),
  include.lowest = TRUE,
  labels = c(1,2,3,4)
)
```

On découpe la population en 4 groupes selon les quartiles, ce qui nous donne des strates de taille à peu près égale.

Le nombre de communes par strate et la taille d'échantillon allouée à chaque strate

Résultats avec 4 strates

	▲ strate ▼	effectif ▼
1	1	322
2	2	322
3	3	321
4	4	322

On effectue ensuite le tirage stratifié proportionnel : on tire dans chaque strate un nombre de communes proportionnel à la taille de la strate.

La fonction de tirage stratifié

```
# Table finale stratifiée
datastrat <- donnees[, c("Code.département", "Commune", "Population.totale", "strate")]
head(datastrat)

# On trie les données par strate
data <- datastrat[order(datastrat$strate), ]

# Effectifs par strate
Nh <- table(data$strate)
Nh

# Taille totale
N <- sum(Nh)
|
# Poids des strates
gh <- Nh / N
gh

library(sampling)
n <- 100

# Calcul des tailles proportionnelles
nh <- round(n * Nh / N)
nh

# Ajustement si la somme ≠ 100
sum(nh)

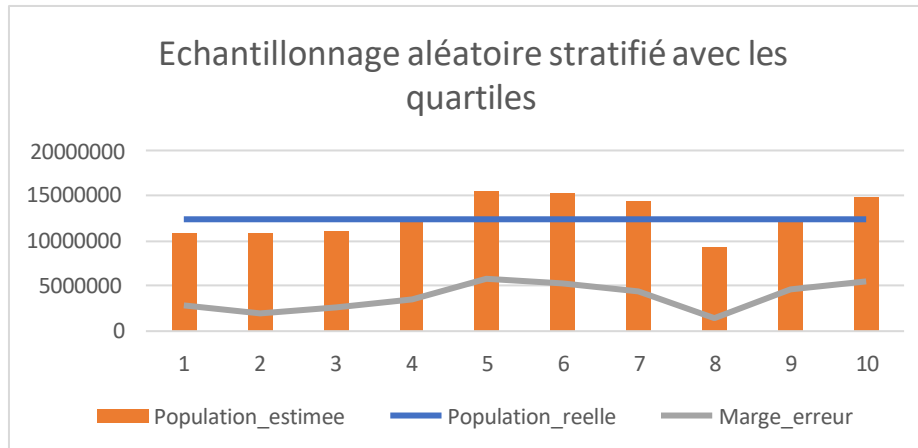
# Exemple de correction si besoin :
nh[which.max(nh)] <- nh[which.max(nh)] + (n - sum(nh))
nh
```

Pour chaque strate on calcule le nombre de communes à tirer (nh) de façon proportionnelle, puis on tire aléatoirement dans chaque strate.

La boucle des 10 tirages stratifiés

```
for(i in 1:10){  
  # Tirage stratifié (comme dans le TP)  
  st <- strata(data, stratanames = c("strate"), size = nh, method = "srswor")  
  data1 <- getdata(data, st)  
  
  head(data1)  
  
  # Sous-échantillons  
  ech1 <- data1[data1$strate == 1, ]  
  ech2 <- data1[data1$strate == 2, ]  
  ech3 <- data1[data1$strate == 3, ]  
  ech4 <- data1[data1$strate == 4, ]  
  ...  
  
  new.row <- data.frame(  
    Population_reelle = T_vrai,  
    Population_estimee = round(T_strat),  
    Marge_erreur = round(marge),  
    IDC_bas = idc_T[1],  
    IDC_haut = idc_T[2]  
  )  
  resultats_strat <- rbind(resultats_strat, new.row)  
}
```

Le graphique des 10 communes pour le sondage stratifié à 4 strates



On peut déjà voir que les intervalles de confiance sont plus resserrés qu'avec le SAS, ce qui montre que le sondage stratifié est plus précis.

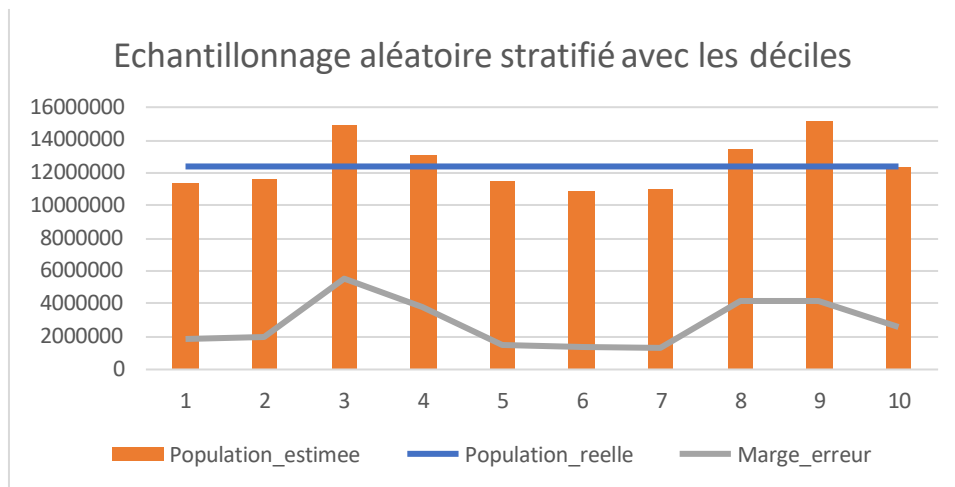
Amélioration avec 10 strates (déciles)

On pousse l'analyse plus loin en passant à **10 strates** basées sur les déciles de population.

Le code de création des 10 strates avec les déciles

```
# Création de 10 strates à partir des déciles
donnees$strate <- cut(
  donnees$Population.totale,
  breaks = quantile(donnees$Population.totale, probs = seq(0, 1, 0.1)),
  include.lowest = TRUE,
  labels = c(1,2,3,4,5,6,7,8,9,10)
```

Le graphique des 10 communes pour le sondage stratifié à 10 strates



On remarque que les intervalles sont encore plus resserrés qu'avec 4 strates.

L'étude comparative entre le sondage aléatoire simple et le sondage stratifié montre que les deux méthodes permettent d'estimer correctement la population totale de l'Île-de-France, puisque la vraie valeur est généralement incluse dans les intervalles de confiance.

Cependant, le sondage stratifié apparaît globalement plus performant et précis. En effet, les marges d'erreur sont en moyenne plus faibles, ce qui traduit une meilleure précision des estimations. Cela s'explique par le fait que la stratification permet de regrouper les communes en sous-ensembles homogènes selon leur taille de population, réduisant ainsi la variabilité à l'intérieur de chaque groupe.

Le sondage aléatoire simple, quant à lui, ne tient pas compte de cette structure et peut conduire à des estimations plus dispersées selon les tirages.

Ainsi, le sondage stratifié est plus adapté lorsque la population est hétérogène, comme c'est le cas pour les communes d'Île-de-France.

Conclusion de la Partie 1

En résumé, on a pu voir que le **sondage aléatoire simple** est facile à mettre en place mais manque de précision lorsque la population est hétérogène. Le **sondage stratifié** permet d'améliorer cette précision en s'assurant que toutes les catégories de communes sont représentées.

Partie 2 — Enquête Sport & Étudiants

2.1 Présentation des données

Dans cette deuxième partie, on travaille sur une enquête portant sur la pratique sportive des étudiants. Le fichier contient plusieurs variables qualitatives comme le sexe, le niveau d'études, ou encore la filière. L'objectif est de voir si certaines de ces variables sont liées à la pratique du sport.

Le code de chargement des données sport

```
# Import du fichier
donneesPartie2 = read.csv2("EnqueteSportEtudiant2024.csv")
```

Un aperçu du jeu de données

Cette capture permet de voir les premières variables disponibles dans le fichier.

```
> # Structure des données
> str(donneesPartie2)
'data.frame': 375 obs. of 76 variables:
 $ sexe           : chr  "un homme" "un homme" "un homme" "un homme" ...
 $ deptgeo        : chr  "17" "17" "79" "Autre" ...
 $ deptgeo_Autre  : chr  "" "" "" "49" ...
 $ deptformation  : chr  "SD" "SD" "SD" "SD" ...
 $ niveau         : chr  "BUT3" "BUT3" "BUT3" "BUT3" ...
 $ reprise        : chr  "Non" "Non" "Non" "Non" ...
 $ alternant      : chr  "Non" "Oui" "Oui" "Oui" ...
 $ bourse         : chr  "Non" "" "" "" ...
 $ travail        : chr  "Non" "" "" "" ...
 $ logement       : chr  "Locataire" "Locataire" "Domicili\xe9 chez vos (ou un de vos ) parents" "Locataire" ...
 $ logement_Autre : chr  "" "" "" "" ...
 $ fumer          : chr  "Non" "Non" "Non" "Oui" ...
```

2.2 Tableaux croisés et tests du Khi-deux

Pour chaque variable, on construit un **tableau croisé** avec la pratique du sport, puis on réalise un **test du Khi-deux** pour voir si la liaison est statistiquement significative.

Le code de construction d'un tableau croisé et du test Khi-deux pour une variable (exemple : le sexe)

```
# Tableau croisé sport / sexe
tab_sexe<- subset(donneesPartie2, !is.na(sexe) & !is.na(sport) & sport != "" & sexe != "")
tab_sexe <- table(tab_sexe$sexe, tab_sexe$sport)
tab_sexe
prop.table(tab_sexe, margin=2)
```

```
# Tests du khi-deux
test_sexe = chisq.test(tab_sexe)
test_sexe
test_sexe$residuals
test_sexe$residuals^2
test_sexe$expected
```

si la p-valeur est inférieure à 0.05, on rejette l'hypothèse d'indépendance et on conclut qu'il y a une liaison significative.

Le résultat du test Khi-deux pour le sport et le sexe

```
data: tab_sexe
X-squared = 13.302, df = 1, p-value = 0.0002651
```

2.3 V de Cramer et tableau récapitulatif

Le test du Khi-deux nous dit si une liaison existe, mais pas à quel point elle est forte. Pour ça, on utilise le **V de Cramer**, qui est un indicateur compris entre 0 et 1 : plus il est proche de 1, plus la liaison est forte.

Le code de calcul du V de Cramer

```
# Exemple Calcul du V de Cramer avec la formule vu en TD

Paramètres
n <- sum(tab_sexe)
p <- nrow(tab_sexe)
q <- ncol(tab_sexe)
m <- min(p - 1, q - 1)

# V de Cramer
V_sexe <- sqrt(test_sexe$statistic / (n * m))

V_sexe

#ou alors on peut aussi on le fait de manière plus simple grâce à la librairie "lsr"
cramersv(tab_sexe)
```

Le code d'un exemple de calcul du khi-deux et v de Cramer

```
# Tests du khi-deux
test_sexe = chisq.test(tab_sexe)
test_sexe
test_sexe$residuals
test_sexe$residuals^2
test_sexe$expected

...

# Exemple calcul du v de Cramer avec la formule vu en TD

#Paramètres
n <- sum(tab_sexe)
p <- nrow(tab_sexe)
q <- ncol(tab_sexe)
m <- min(p - 1, q - 1)

# V de Cramer
V_sexe <- sqrt(test_sexe$statistic / (n * m))

V_sexe
```

Le tableau récapitulatif avec toutes les variables, leur p-valeur et leur V de Cramer

Variable	P_value	V_de_Cramer
Sexe	0,000265143	0,188590764
Niveau	0,0431137	0,147769053
Bourse	0,170238758	0,089845828
Logement	0,215574734	0,064731224
Travail	0,720390295	0,023449247

Ce tableau est le résultat central de cette partie. Pour chaque variable, on voit si la relation avec le sport est significative et quelle est l'intensité de cette liaison.

Les analyses statistiques révèlent que seules les variables Sexe et Niveau présentent une relation statistiquement significative avec la variable étudiée ($p < 0,05$). Pour les variables Bourse, Logement et Travail, les p-valeurs supérieures à 0,05 indiquent que les faibles associations observées par le V de Cramer ne peuvent pas être distinguées du hasard ; elles sont donc considérées comme non significatives.

2.4 Commentaires et conclusion

En regardant les résultats, on peut voir que certaines variables sont plus liées à la pratique du sport que d'autres. La variable avec le V de Cramer le plus élevé est celle qui a la liaison la plus forte avec le sport.

À l'inverse, certaines variables n'ont pas de lien significatif avec la pratique sportive, ce qui veut dire que ces facteurs n'ont pas vraiment d'influence sur le fait de faire du sport ou non chez les étudiants interrogés.

Conclusion générale

Ce travail nous a permis de mettre en pratique des notions importantes de statistique inférentielle. On a vu que l'échantillonnage est une méthode puissante pour estimer une grandeur dans une population, mais que la précision dépend beaucoup de la méthode choisie.

Le sondage aléatoire simple est simple à mettre en place mais peut manquer de précision quand la population est hétérogène, comme c'est le cas en Île-de-France. Le sondage stratifié règle ce problème en s'assurant que toutes les catégories sont bien représentées.

Dans la deuxième partie, on a appris à utiliser le test du Khi-deux et le V de Cramer pour détecter et mesurer des relations entre variables qualitatives. Ces outils sont vraiment utiles pour analyser des données d'enquête et tirer des conclusions sur les comportements d'une population.

CODE R :

```
setwd("...")

#Partie 1

# On importe le fichier CSV (après suppression des 6 premières lignes)
# dans Excel et sauvegarde au format .csv)
population <- read.csv2("population_francaise_communes.csv", encoding = "latin1", skip = 7)

# On vérifie la structure du fichier importé
str(population)
head(population)

# On affiche les noms des colonnes pour vérifier
colnames(population)
# On filtre uniquement les communes de la région Île-de-France
# et on sélectionne les 3 colonnes utiles : Code.département, Commune, Population.totale

donnees <- population[population$Nom.de.la.région == "Île-de-France",
  c("Code.département", "Commune", "Population.totale")]

# On réinitialise les indices des lignes
rownames(donnees) <- NULL

# Affichage des 6 premières lignes
head(donnees)

# Vérification : on s'assure qu'il n'y a pas de valeurs manquantes
sum(is.na(donnees))

# La population U est l'ensemble des communes d'Île-de-France
# Chaque commune est un "individu" dans notre population

# On crée le vecteur U qui stockera toutes les communes
U <- donnees$Commune

# Nombre total N de communes en Île-de-France
N <- length(U)
cat("Nombre total de communes en Île-de-France : N =", N, "\n")

# T est la somme de la population totale de toutes les communes
# d'Île-de-France : c'est la vraie valeur que l'on cherchera à estimer

donnees$Population.totale <- as.numeric(gsub(" ", "", donnees$Population.totale))
T_vrai <- sum(donnees$Population.totale)
cat("Nombre exact d'habitants en Île-de-France : T =", T_vrai, "\n")

resultats <- data.frame(
  Population_reelle = numeric(),
  Population_estimee = numeric(),
  Marge_erreur = numeric(),
  IDC_bas = numeric(),
  IDC_haut = numeric()
)
for (i in 1:10) {

# Taille de l'échantillon
n <- 100

# Tirage aléatoire simple (SAS) sans remise de n=100 communes
# parmi les N communes de la population U
E <- sample(1:nrow(donnees), n)

# Création de la table "donnees1" :
# On récupère les données des communes sélectionnées
# avec leurs 3 colonnes : Commune, Code.département, Population.totale

donnees1 <- donnees[E,
  c("Code.département", "Commune", "Population.totale")]

# Réinitialisation des indices
rownames(donnees1) <- NULL

# Affichage des 6 premières lignes de l'échantillon
head(donnees1)

cat("Taille de l'échantillon :", nrow(donnees1), "\n")

# Calcul de la moyenne d'habitants dans l'échantillon
xbar <- mean(donnees1$Population.totale)
cat("Moyenne d'habitants par commune (échantillon) :", xbar, "\n")

# Intervalle de confiance à 95% pour la moyenne  $\mu$ 
# On utilise le test t de Student (t.test) qui fournit l'IDC
idc_moy <- t.test(donnees1$Population.totale, conf.level = 0.95)$conf.int
```



```

cat("IDC à 95% pour  $\mu$  : [", idc_moy[1], ";", idc_moy[2], "]\n")

# Estimation du total T à partir de l'échantillon :
# T_est = N * xbar (on extrapole la moyenne à toute la population)
T_est <- N * xbar
cat("Estimation du nombre total d'habitants : T_est =", T_est, "\n")
cat("Vraie valeur T =", T_vrai, "\n")
cat("Différence (T - T_est) =", T_vrai - T_est, "\n")

# IDC pour T : on multiplie les bornes de l'IDC de  $\mu$  par N
idc_T <- idc_moy * N
cat("IDC à 95% pour T : [", idc_T[1], ";", idc_T[2], "]\n")

# Marge d'erreur = demi-largeur de l'intervalle
marge <- (idc_T[2] - idc_T[1]) / 2
cat("Marge d'erreur :", marge, "\n")

# On vérifie si la vraie valeur T est bien dans l'intervalle
if (T_vrai >= idc_T[1] & T_vrai <= idc_T[2]) {
  cat("✓ La vraie valeur T est DANS l'intervalle de confiance.\n")
} else {
  cat("✗ La vraie valeur T est HORS de l'intervalle de confiance.\n")
}
new.row <- data.frame(
  Population_reelle = T_vrai,
  Population_estimee = round(T_est),
  Marge_erreur = round(marge),
  IDC_bas = idc_T[1],
  IDC_haut = idc_T[2]
)

resultats <- rbind(resultats, new.row)
}

print(resultats)
write.csv2(resultats, "resultatsEchantillon.csv", row.names = FALSE)

#####

# On regarde la distribution de la population
summary(donnees$Population.totale)

# Création de 4 strates à partir des quantiles
donnees$strate <- cut(
  donnees$Population.totale,
  breaks = quantile(donnees$Population.totale, probs = seq(0, 1, 0.25)),
  include.lowest = TRUE,
  labels = c(1,2,3,4)
)

# Table finale stratifiée
datastrat <- donnees[, c("Code.département", "Commune", "Population.totale", "strate")]
head(datastrat)

# On trie les données par strate
data <- datastrat[order(datastrat$strate), ]

# Effectifs par strate
Nh <- table(data$strate)
Nh

# Taille totale
N <- sum(Nh)

# Poids des strates
gh <- Nh / N
gh

library(sampling)
n <- 100

# Calcul des tailles proportionnelles
nh <- round(n * Nh / N)
nh

# Ajustement si la somme ≠ 100
sum(nh)

# Exemple de correction si besoin :
nh[which.max(nh)] <- nh[which.max(nh)] + (n - sum(nh))
nh

resultats_strat <- data.frame()
for(i in 1:10){
  # Tirage stratifié (comme dans le TP)
  st <- strata(data, stratanames = c("strate"), size = nh, method = "srswor")
  data1 <- getdata(data, st)

```

```

head(data1)

# Sous-échantillons
ech1 <- data1[data1$strate == 1, ]
ech2 <- data1[data1$strate == 2, ]
ech3 <- data1[data1$strate == 3, ]
ech4 <- data1[data1$strate == 4, ]

# Moyennes
m1 <- mean(ech1$Population.totale)
m2 <- mean(ech2$Population.totale)
m3 <- mean(ech3$Population.totale)
m4 <- mean(ech4$Population.totale)

# Variances
var1 <- var(ech1$Population.totale)
var2 <- var(ech2$Population.totale)
var3 <- var(ech3$Population.totale)
var4 <- var(ech4$Population.totale)

# Moyenne stratifiée
Xbarst <- (Nh[1]*m1 + Nh[2]*m2 + Nh[3]*m3 + Nh[4]*m4) / N

# Fractions de sondage
fh <- nh / Nh

# Variance de Xbarst (formule du TP)
varXbarst <- (gh[1]^2)*(1 - fh[1])*var1/nh[1] +
  (gh[2]^2)*(1 - fh[2])*var2/nh[2] +
  (gh[3]^2)*(1 - fh[3])*var3/nh[3] +
  (gh[4]^2)*(1 - fh[4])*var4/nh[4]
alpha <- 0.05

binf <- Xbarst - qnorm(1 - alpha/2) * sqrt(varXbarst)
bsup <- Xbarst + qnorm(1 - alpha/2) * sqrt(varXbarst)

idc_moy <- c(binf, bsup)

cat("IDC pour la moyenne :", idc_moy, "\n")

# Estimation du total
T_strat <- N * Xbarst

# Intervalle de confiance
idc_T <- idc_moy * N

# Marge d'erreur
marge <- (idc_T[2] - idc_T[1]) / 2

cat("Estimation stratifiée :", T_strat, "\n")
cat("Vraie valeur :", T_vrai, "\n")
cat("IDC :", idc_T, "\n")
cat("Marge d'erreur :", marge, "\n")

new.row <- data.frame(
  Population_reelle = T_vrai,
  Population_estimee = round(T_strat),
  Marge_erreur = round(marge),
  IDC_bas = idc_T[1],
  IDC_haut = idc_T[2]
)
resultats_strat <- rbind(resultats_strat, new.row)
}

print(resultats_strat)
write.csv2(resultats_strat, "resultats_strat.csv", row.names = FALSE)
#####
# On regarde la distribution de la population
summary(donnees$Population.totale)

# Création de 4 strates à partir des déciles
donnees$strate <- cut(
  donnees$Population.totale,
  breaks = quantile(donnees$Population.totale, probs = seq(0, 1, 0.1)),
  include.lowest = TRUE,
  labels = c(1,2,3,4,5,6,7,8,9,10)
)

# Table finale stratifiée
datastrat <- donnees[, c("Code.département", "Commune", "Population.totale", "strate")]

head(datastrat)

# On trie les données par strate
data <- datastrat[order(datastrat$strate), ]

# Effectifs par strate
Nh <- table(data$strate)
Nh

```

```

# Taille totale
N <- sum(Nh)

# Poids des strates
gh <- Nh / N
gh

library(sampling)
n <- 100

# Calcul des tailles proportionnelles
nh <- round(n * Nh / N)
nh

# Ajustement si la somme ≠ 100
sum(nh)

# Exemple de correction si besoin :
nh[which.max(nh)] <- nh[which.max(nh)] + (n - sum(nh))
nh

resultats_strat <- data.frame()
for(i in 1:10){
  # Tirage stratifié (comme dans le TP)
  st <- strata(data, stratanames = c("strate"), size = nh, method = "srswor")
  data1 <- getdata(data, st)

  head(data1)

  # Sous-échantillons
  ech1 <- data1[data1$strate == 1, ]
  ech2 <- data1[data1$strate == 2, ]
  ech3 <- data1[data1$strate == 3, ]
  ech4 <- data1[data1$strate == 4, ]
  ech5 <- data1[data1$strate == 5, ]
  ech6 <- data1[data1$strate == 6, ]
  ech7 <- data1[data1$strate == 7, ]
  ech8 <- data1[data1$strate == 8, ]
  ech9 <- data1[data1$strate == 9, ]
  ech10 <- data1[data1$strate == 10, ]

  # Moyennes
  m1 <- mean(ech1$Population.totale)
  m2 <- mean(ech2$Population.totale)
  m3 <- mean(ech3$Population.totale)
  m4 <- mean(ech4$Population.totale)
  m5 <- mean(ech5$Population.totale)
  m6 <- mean(ech6$Population.totale)
  m7 <- mean(ech7$Population.totale)
  m8 <- mean(ech8$Population.totale)
  m9 <- mean(ech9$Population.totale)
  m10 <- mean(ech10$Population.totale)

  # Variances
  var1 <- var(ech1$Population.totale)
  var2 <- var(ech2$Population.totale)
  var3 <- var(ech3$Population.totale)
  var4 <- var(ech4$Population.totale)
  var5 <- var(ech5$Population.totale)
  var6 <- var(ech6$Population.totale)
  var7 <- var(ech7$Population.totale)
  var8 <- var(ech8$Population.totale)
  var9 <- var(ech9$Population.totale)
  var10 <- var(ech10$Population.totale)

  # Moyenne stratifiée
  Xbarst <- (Nh[1]*m1 + Nh[2]*m2 + Nh[3]*m3 + Nh[4]*m4 + Nh[5]*m5 + Nh[6]*m6 + Nh[7]*m7 + Nh[8]*m8 + Nh[9]*m9 + Nh[10]*m10) / N

  # Fractions de sondage
  fh <- nh / Nh

  # Variance de Xbarst (formule du TP)
  varXbarst <- (gh[1]^2)*(1 - fh[1])*var1/nh[1] +
    (gh[2]^2)*(1 - fh[2])*var2/nh[2] +
    (gh[3]^2)*(1 - fh[3])*var3/nh[3] +
    (gh[4]^2)*(1 - fh[4])*var4/nh[4] +
    (gh[5]^2)*(1 - fh[5])*var5/nh[5] +
    (gh[6]^2)*(1 - fh[6])*var6/nh[6] +
    (gh[7]^2)*(1 - fh[7])*var7/nh[7] +
    (gh[8]^2)*(1 - fh[8])*var8/nh[8] +
    (gh[9]^2)*(1 - fh[9])*var9/nh[9] +
    (gh[10]^2)*(1 - fh[10])*var10/nh[10]

  alpha <- 0.05

  binf <- Xbarst - qnorm(1 - alpha/2) * sqrt(varXbarst)

```

```

bsup <- Xbarst + qnorm(1 - alpha/2) * sqrt(varXbarst)

idc_moy <- c(binf, bsup)

cat("IDC pour la moyenne :", idc_moy, "\n")

# Estimation du total
T_strat <- N * Xbarst

# Intervalle de confiance
idc_T <- idc_moy * N

# Marge d'erreur
marge <- (idc_T[2] - idc_T[1]) / 2

cat("Estimation stratifiée :", T_strat, "\n")
cat("Vraie valeur :", T_vrai, "\n")
cat("IDC :", idc_T, "\n")
cat("Marge d'erreur :", marge, "\n")

new.row <- data.frame(
  Population_reelle = T_vrai,
  Population_estimee = round(T_strat),
  Marge_erreur = round(marge),
  IDC_bas = idc_T[1],
  IDC_haut = idc_T[2]
)
resultats_strat <- rbind(resultats_strat, new.row)
}

print(resultats_strat)
write.csv2(resultats_strat, "resultats_stratendecile.csv", row.names = FALSE)

#####
#Partie 2
# Charger la librairie
library(lsr)

# Import du fichier
donneesPartie2 = read.csv2("EnqueteSportEtudiant2024.csv")

# Afficher les 6 premières lignes
head(donneesPartie2)

# Structure des données
str(donneesPartie2)

# Dimensions
dim(donneesPartie2)

# Tableau croisé sport / sexe
tab_sexe <- subset(donneesPartie2, !is.na(sexe) & !is.na(sport) & sport != "" & sexe != "")
tab_sexe <- table(tab_sexe$sexe, tab_sexe$sport)
tab_sexe
prop.table(tab_sexe, margin=2)

# Tableau croisé sport / niveau
tab_niveau <- subset(donneesPartie2, !is.na(niveau) & !is.na(sport) & sport != "" & niveau != "")
tab_niveau <- table(tab_niveau$niveau, tab_niveau$sport)
tab_niveau
prop.table(tab_niveau, margin=2)

# Tableau croisé sport / bourse
tab_bourse <- subset(donneesPartie2, !is.na(bourse) & !is.na(sport) & sport != "" & bourse != "")
tab_bourse <- table(tab_bourse$bourse, tab_bourse$sport)
tab_bourse
prop.table(tab_bourse, margin=2)

# Tableau croisé sport / logement
tab_logement <- subset(donneesPartie2, !is.na(logement) & !is.na(sport) & sport != "" & logement != "" & logement != "Autre")
tab_logement <- table(tab_logement$logement, tab_logement$sport)
tab_logement
prop.table(tab_logement, margin=2)

# Tableau croisé sport / travail
tab_travail <- subset(donneesPartie2, !is.na(travail) & !is.na(sport) & sport != "" & travail != "")
tab_travail <- table(tab_travail$travail, tab_travail$sport)
tab_travail
prop.table(tab_travail, margin=2)

# Tests du khi-deux
test_sexe = chisq.test(tab_sexe)
test_sexe
test_sexe$residuals
test_sexe$residuals^2
test_sexe$expected

test_niveau = chisq.test(tab_niveau)
test_niveau

```

```

test_niveau$residuals
test_niveau$residuals^2
test_niveau$expected

test_bourse = chisq.test(tab_bourse)
test_bourse
test_bourse$residuals
test_bourse$residuals^2
test_bourse$expected

test_logement = chisq.test(tab_logement)
test_logement
test_logement$residuals
test_logement$residuals^2
test_logement$expected

test_travail = chisq.test(tab_travail)
test_travail
test_travail$residuals
test_travail$residuals^2
test_travail$expected

# Exemple Calcul du V de Cramer avec la formule vu en TD

# Paramètres
n <- sum(tab_sexe)
p <- nrow(tab_sexe)
q <- ncol(tab_sexe)
m <- min(p - 1, q - 1)

# V de Cramer
V_sexe <- sqrt(test_sexe$statistic / (n * m))

V_sexe

# OU alors on peut aussi on le fait de manière plus simple grâce à la librairie "lsr"
cramersV(tab_sexe)

cramersV(tab_niveau)

cramersV(tab_bourse)

cramersV(tab_logement)

cramersV(tab_travail)

# Création du tableau récapitulatif
resultats_tests <- data.frame(
  Variable = c("Sexe", "Niveau", "Bourse", "Logement", "Travail"),

  P_value = c(
    test_sexe$p.value,
    test_niveau$p.value,
    test_bourse$p.value,
    test_logement$p.value,
    test_travail$p.value
  ),

  V_de_Cramer = c(
    crammersV(tab_sexe),
    crammersV(tab_niveau),
    crammersV(tab_bourse),
    crammersV(tab_logement),
    crammersV(tab_travail)
  )
)

# Affichage
print(resultats_tests)

# Export CSV
write.csv2(resultats_tests, "resultats_khi2_cramer.csv", row.names = FALSE)

test <- data.frame(table(datastrat$strate))
colnames(test) <- c("strate", "effectif")

```

